

PBA: 7. Summarizing Bivariate Relationships & Tidying Data

Fall 2025

Jaewon (“Jay-one”) Yoo

National Tsing Hua University

Roadmap

- 0. Causality review
- 1. Z-scores and standardization
- 2. Correlation
- 3. Writing our own functions
- 4. Pivoting data longer
- 5. Joining data sets

o/ Causality review

Potential Outcomes *Redux*



- Potential outcomes:
 - $Y_i(1)$ is the value that the outcome would be if we gave unit i treatment and changed nothing else about them.
 - $Y_i(0)$ is the value that the outcome would be if we gave unit i **no treatment** and changed nothing else about them.
 - Not the **possible values** of the outcome.

COVID-19 Vaccine Trials



- **Treatment:** $T_i = 1$ if vaccinated, $T_i = 0$ if not.
- **Outcome:** $Y_i = 1$ if acquired COVID after 12 weeks, $Y_i = 0$ if not.
 - Q1. What are the potential outcomes $Y_i(1)$ and $Y_i(0)$?
 - Q2. Why not compare early volunteers for the vaccine with the overall population?

1/ Z-scores and standardization

Recall: C19 Vaccination Status in Mid 2021



Source: <https://bit.ly/47jguJE>

- By mid-2021, slightly above 60% of the population in the US had received at least one dose of a COVID-19 vaccine.

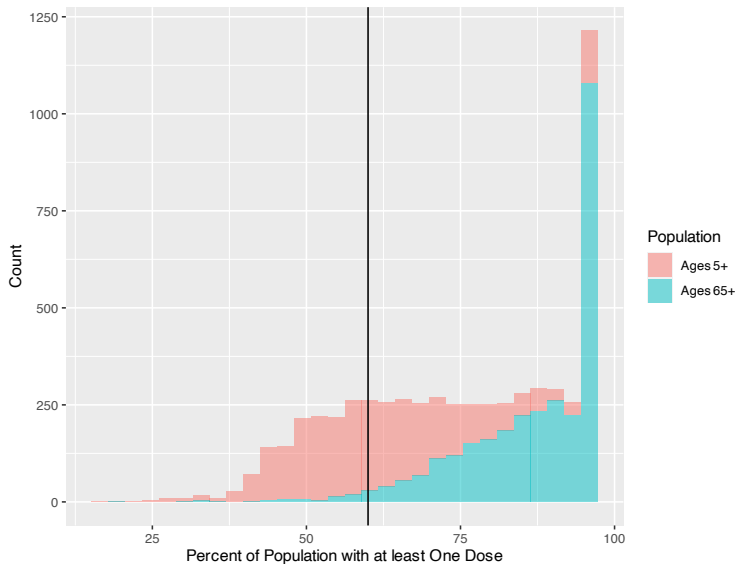
⇒ **Q: Is 60% vaccinated a lot?**

COVID Vaccination Data

- Data on county-level vaccination rates and presidential voting:

```
1 # Don't forget to load tidyverse and ggplot2!
2 > covid_data <- as_tibble(read.csv(url("https://bit.ly/3F0Lohq"))); covid_data
3
4 # A tibble: 3,114 × 8
5   fips county state one_dose_5plus_pct one_dose_65plus_pct booster_5plus_pct dem_pct_2000
6   <chr> <chr> <chr>           <dbl>           <dbl>           <dbl>           <dbl>
7 1 26039 Crawf. MI             55.7             77.3             31.2             43.8
8 2 40015 Caddo. OK              83.3             95              30.3             46.4
9 3 17007 Boone. IL             71.1             94.5             35.1             41.8
10 4 12055 Highl. FL             68.9             93.7             24.7             40.3
11 5 34029 Ocean. NJ             71              95              32.1             47.2
12 6 01067 Henry. AL             58.5             85.5             18.2             40.1
13 7 27037 Dakot. MN             81              95              49.5             46.9
14 8 27115 Pine . MN             56.5             85              31.7             47.0
15 9 51750 Radfo. VA             41.5             73.8             1.79             46.4
16 10 22009 Avoye. LA            59.7             80.1             21.9             45.7
17 # 3,104 more rows
18 # 1 more variable: dem_pct_2020 <dbl>
19 # Use `print(n = ...)` to see more rows
```

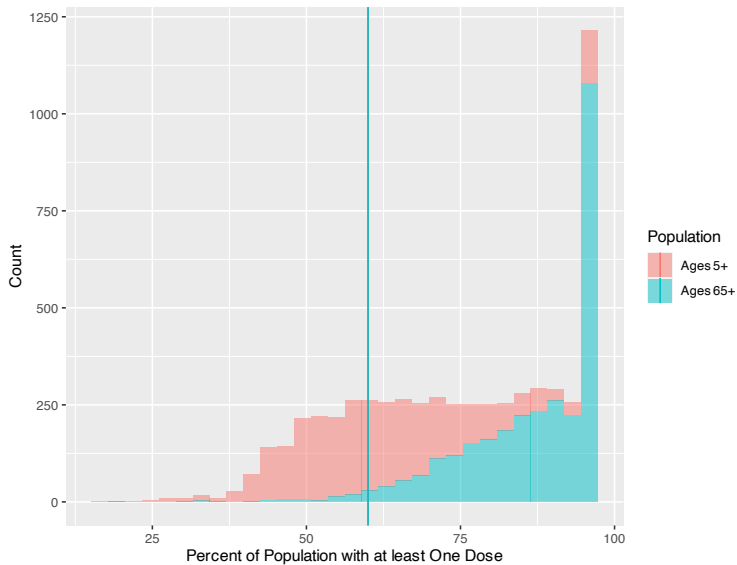
Is 60% Vaccinated A Lot?



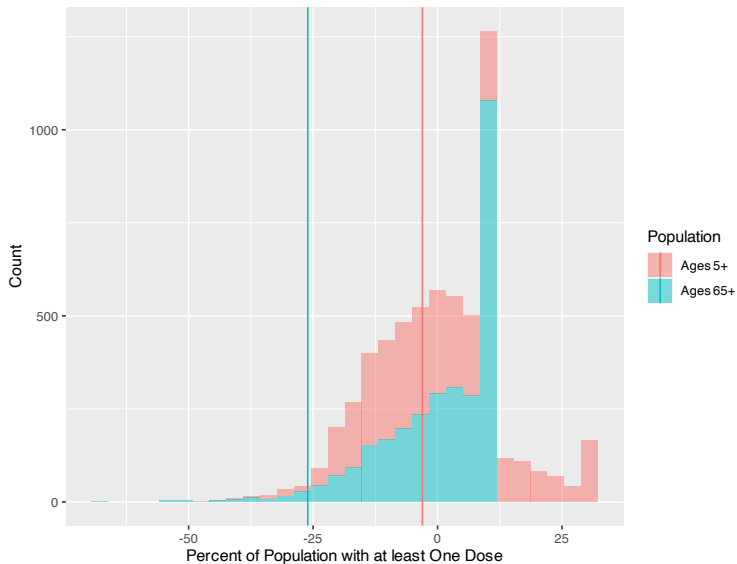
How Large is Large?

- The answer hinges on where the value (60%) stands in relation to the distribution.
 - A histogram can give a visual clue, but is that enough for precise comparison?
 - Middling for the 5+ group, but very low for the 65+ group.
- Can we transform the values of our variables to be in **common units**?
- Two transformations:
 - **Centering**: subtract the mean of the variable from each value.
 - **Scaling**: dividing deviations from the mean by the standard deviation.

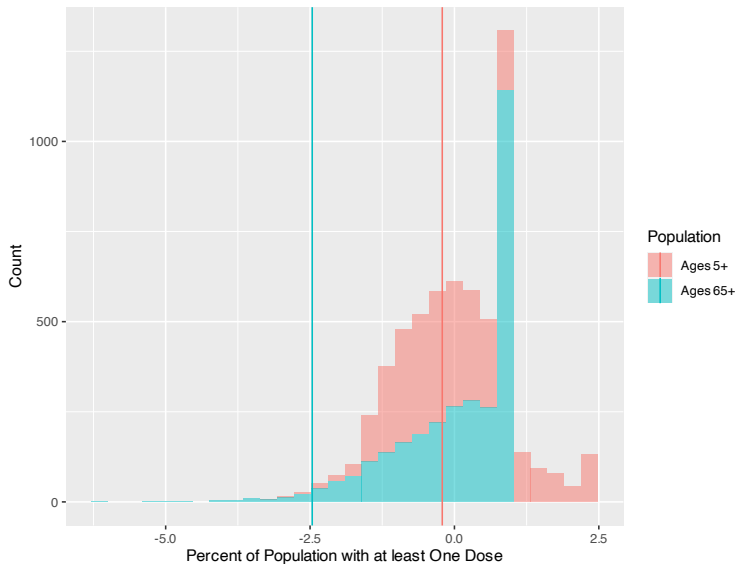
Original Distributions



Centered Distributions



Centered and Scaled Distributions



Z-Scores

- Centering tells us immediately if a value is above or below the mean.
- Scaling tells us how many standard deviations away from the mean it is.
- Combine them with the **z-score** transformation:

$$\text{z-score of } x_i = \frac{x_i - \text{mean of } x}{\text{standard deviation of } x}$$

- Useful heuristic: data more than 3 SDs away from the mean are 'rare.'

Z-Score Example: Centering

```
1 > covid_data |>
2   mutate(one_dose_centered = one_dose_5plus_pct -
3     mean(one_dose_5plus_pct, na.rm = T)) |>
4   select(fips:state, one_dose_5plus_pct, one_dose_centered)
5
6 # A tibble: 3,114 × 5
7   fips county      state one_dose_5plus_pct one_dose_centered
8   <int> <chr>      <chr>          <dbl>          <dbl>
9 1 26039 Crawford County MI              55.7           -7.35
10 2 40015 Caddo County OK              83.3            20.2
11 3 17007 Boone County IL              71.1            8.05
12 4 12055 Highlands County FL              68.9            5.85
13 5 34029 Ocean County NJ              71              7.95
14 6 1067 Henry County AL              58.5           -4.55
15 7 27037 Dakota County MN              81             17.9
16 8 27115 Pine County MN              56.5           -6.55
17 9 51750 Radford city VA              41.5          -21.6
18 10 22009 Avoyelles Parish LA              59.7           -3.35
19 # 3,104 more rows
20 # Use `print(n = ...)` to see more rows
```

Z-Score Example: Centering & Scaling

```
1 > covid_data |>
2   mutate(one_dose_z = (one_dose_5plus_pct - mean(one_dose_5plus_pct, na.rm = T)) /
3     sd(one_dose_5plus_pct, na.rm = T)) |>
4   select(fips:state, one_dose_5plus_pct, one_dose_z)
5
6 # A tibble: 3,114 × 5
7   fips county      state one_dose_5plus_pct one_dose_z
8   <int> <chr>      <chr>          <dbl>      <dbl>
9 1 26039 Crawford County MI              55.7      -0.508
10 2 40015 Caddo County OK              83.3       1.40
11 3 17007 Boone County IL              71.1       0.556
12 4 12055 Highlands County FL              68.9       0.404
13 5 34029 Ocean County NJ              71          0.549
14 6 1067 Henry County AL              58.5      -0.314
15 7 27037 Dakota County MN              81         1.24
16 8 27115 Pine County MN              56.5      -0.452
17 9 51750 Radford city VA              41.5      -1.49
18 10 22009 Avoyelles Parish LA              59.7      -0.231
19 # 3,104 more rows
20 # Use `print(n = ...)` to see more rows
```

- Q: Now, according to our data, is 60% vaccinated a lot?

2/ Correlation

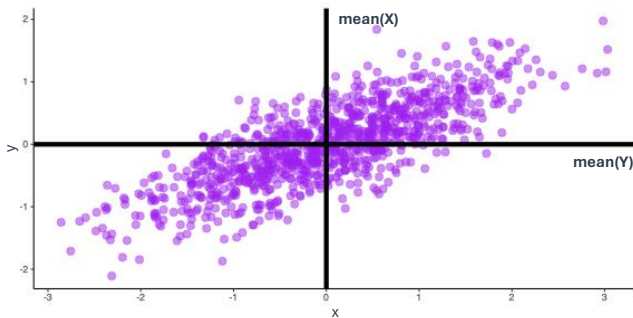
Correlation

- How do variables move together on average?
- When x_i is big, what is y_i likely to be?
 - Positive correlation: when x_i is big, y_i is also big.
 - Negative correlation: when x_i is big, y_i is small.
 - High magnitude of correlation: data cluster tightly around a line.
- The technical definition of the correlation coefficient:

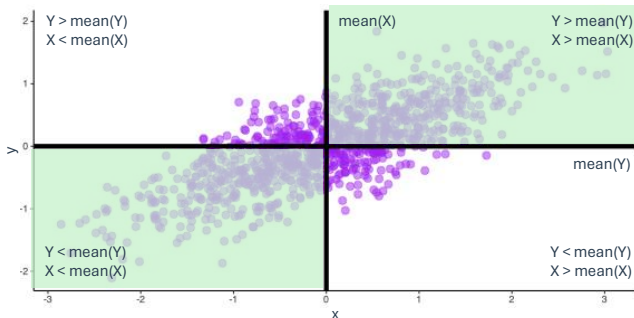
$$\frac{1}{n-1} \sum_{i=1}^n [(z\text{-score for } x_i) \times (z\text{-score for } y_i)]$$

- Interpretation:
 - Correlation is between -1 and 1.
 - Correlation of 0 means no linear association.
 - Positive correlations $\leftrightarrow$ positive associations.
 - Negative correlations $\leftrightarrow$ negative associations.
 - Closer to -1 or 1 means stronger association.

Correlation Intuition

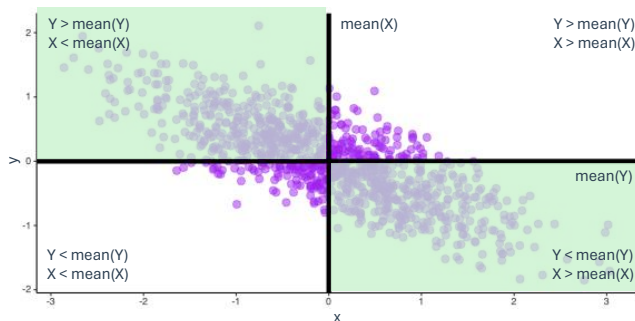


Correlation Intuition



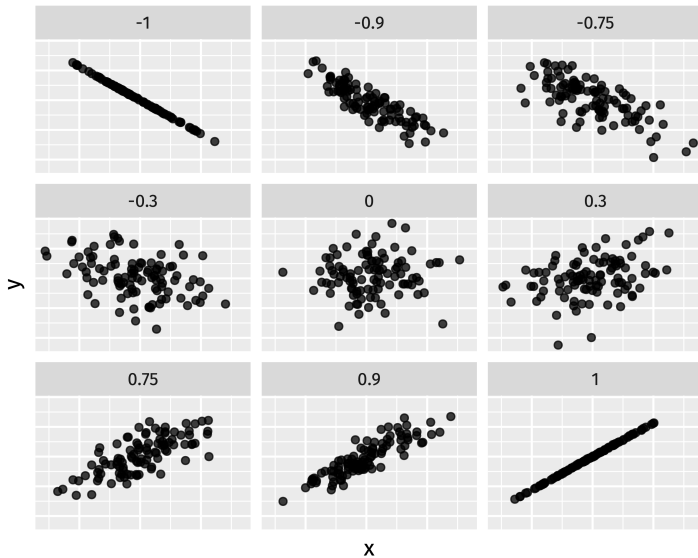
- Large values of X tend to occur with large values of Y:
 - $(\text{z-score for } x_i) \times (\text{z-score for } y_i) = (\text{pos. num.}) \times (\text{pos. num.}) = +$
- Small values of X tend to occur with small values of Y:
 - $(\text{z-score for } x_i) \times (\text{z-score for } y_i) = (\text{neg. num.}) \times (\text{neg. num.}) = +$
- If these dominate $\rightarrow$ **positive correlation**

Correlation Intuition



- Large values of X tend to occur with small values of Y:
 - $(\text{z-score for } x_i) \times (\text{z-score for } y_i) = (\text{pos. num.}) \times (\text{neg. num.}) = -$
- Small values of X tend to occur with large values of Y:
 - $(\text{z-score for } x_i) \times (\text{z-score for } y_i) = (\text{neg. num.}) \times (\text{pos. num.}) = -$
- If these dominate $\rightarrow$ **negative correlation**

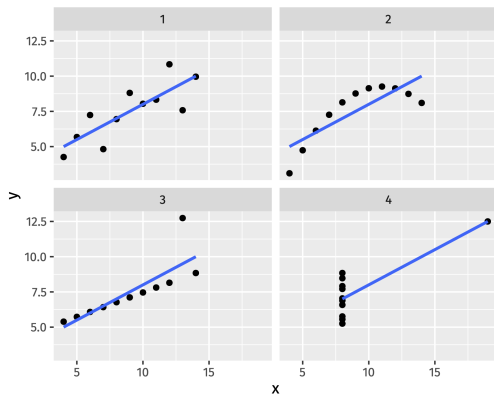
Correlation Examples



Properties of Correlation Coefficient

- Correlation measures **linear** association.
- Order doesn't matter: $\text{cor}(x, y) = \text{cor}(y, x)$
- Not affected by changes of scale:
 - $\text{cor}(x, y) = \text{cor}(ax+b, cy+d)$
 - Celsius vs. Fahrenheit; dollars vs. pesos; cm vs. in.

All 4 Relationships have .816 Correlation



- Francis Anscombe's quartet with identical summary stats:
 - Plot 2: inadequate to capture non-linear relationships
 - Plot 3: sensitive to outlier(s)
 - Plot 4: a single influential point can skew correlation

Correlations in R

- Use the `cor()` function:

```
1 > covid_data |>
2   summarize(cor(one_dose_5plus_pct, dem_pct_2020))
3
4 # A tibble: 1 × 1
5   `cor(one_dose_5plus_pct, dem_pct_2020)`
6                                     <dbl>
7 1                                     NA
```

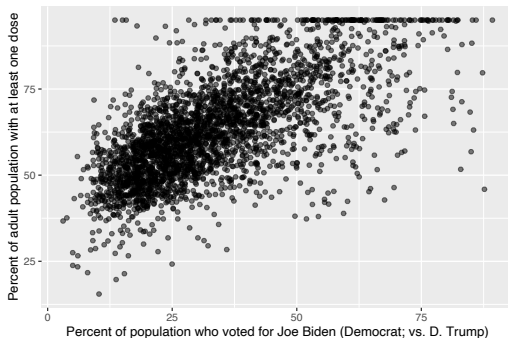
- How should we deal with the missing values?
 - Set the `use = "pairwise"` $\rightsquigarrow$ available case analysis (i.e., we will only use units that have values for both columns.)

```
1 > covid_data |>
2   summarize(cor(one_dose_5plus_pct, dem_pct_2020, use = "pairwise"))
3
4 # A tibble: 1 × 1
5   `cor(one_dose_5plus_pct, dem_pct_2020, use = "pairwise")`
6                                     <dbl>
7 1                                     0.666
```

Comparing Correlations #1

- Is there a relationship between vaccination and political ideology?:

```
1 > covid_data |>  
2   ggplot(aes(x = dem_pct_2020, y = one_dose_5plus_pct)) + geom_point(alpha = .5) +  
3   labs(x = "Percent of population who voted for Joe Biden (Democrat; vs. D. Trump)",  
4        y = "Percent of adult population with at least one dose")
```

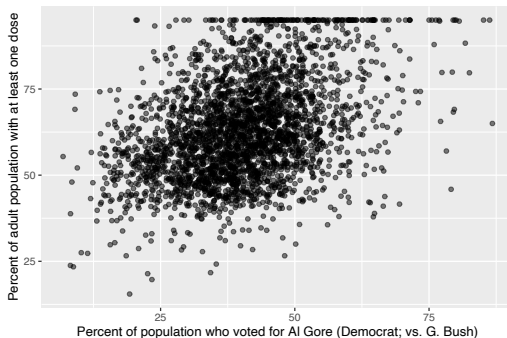


```
1 > cor(covid_data$one_dose_5plus_pct, covid_data$dem_pct_2020, use = "pairwise")  
2 [1] 0.6664387
```

Comparing Correlations #2

- Democrats more prone to vaccinate against COVID 19?:

```
1 > covid_data |>
2   ggplot(aes(x = dem_pct_2000, y = one_dose_5plus_pct)) + geom_point(alpha = .5) +
3   labs(x = "Percent of population who voted for Al Gore (Democrat; vs. G. Bush)",
4        y = "Percent of adult population with at least one dose")
```

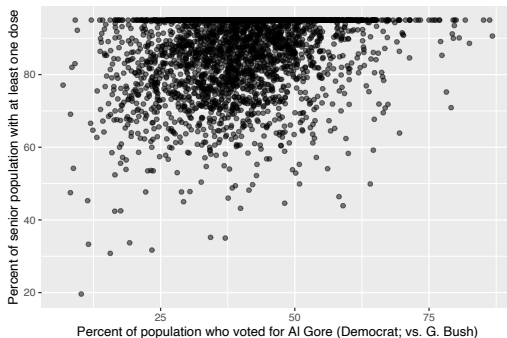


```
1 > cor(covid_data$one_dose_5plus_pct, covid_data$dem_pct_2000, use = "pairwise")
2 [1] 0.3941203
```

Comparing Correlations #3

- Democrats more prone to vaccinate against COVID 19?:

```
1 > covid_data |>  
2   ggplot(aes(x = dem_pct_2000, y = one_dose_65plus_pct)) + geom_point(alpha = .5) +  
3   labs(x = "Percent of population who voted for Al Gore (Democrat; vs. G. Bush)",  
4        y = "Percent of senior population with at least one dose")
```



```
1 > cor(covid_data$one_dose_65plus_pct, covid_data$dem_pct_2000, use = "pairwise")  
2 [1] 0.2631949
```


3/ Writing our own functions

Why Write Functions?

- Supposed we wanted to standardize all the variables in our dataset:
 - Copy-pasting code is tedious and prone to failure!

```
1 > covid_data |>
2   mutate(
3     one_dose_5p_z =
4       (one_dose_5plus_pct - mean(one_dose_5plus_pct, na.rm = TRUE)) /
5       sd(one_dose_5plus_pct, na.rm = TRUE),
6     one_dose_65p_z =
7       (one_dose_65plus_pct - mean(one_dose_65plus_pct, na.rm = TRUE)) /
8       sd(one_dose_65plus_pct, na.rm = TRUE),
9     booster_z =
10      (booster_5plus_pct - mean(booster_5plus_pct, na.rm = TRUE)) /
11      sd(booster_5plus_pct, na.rm = TRUE),
12     dem_pct_2000_z =
13      (dem_pct_2000 - mean(dem_pct_2000, na.rm = TRUE)) /
14      sd(dem_pct_2000, na.rm = TRUE),
15     dem_pct_2020_z =
16      (dem_pct_2020 - mean(dem_pct_2020, na.rm = TRUE)) /
17      sd(dem_pct_2020, na.rm = TRUE)
18   )
```

Writing a New Function

- Notice that all of the mutations follow the same template/pattern:

```
1 > (■■■ - mean(■■■, na.rm = TRUE)) / sd(■■■, na.rm = TRUE)
```

- Only one thing varies: the column of data, represented with ■■■ !

Components of a Function *Redux*

- Recall, we can create functions with the following structure:
 - W6 lecture on User-defined functions in R

```
1 name <- function(arguments) {  
2   body  
3 }
```

- Three components:
 - Name:** the name of the function that we'll use to call it. Maybe `z_score`?
 - Arguments:** things that we want to vary across calls of our function.
We'll use `x`.
 - Body:** the code that the function performs.

Create fn. `z_score()`!

- Convert our template to a function:

```
1 > z_score <- function(x) {  
2   (x - mean(x, na.rm = TRUE)) / sd(x, na.rm = TRUE)  
3 }
```

- Check if it works:

```
1 > z_score(c(1,2,3,4,5))  
2 [1] -1.2649111 -0.6324555  0.0000000  0.6324555  1.2649111
```

Let's Clean Up the Mess

```
1 > covid_data |>
2   mutate(
3     one_dose_5p_z = z_score(one_dose_5plus_pct),
4     one_dose_65p_z = z_score(one_dose_65plus_pct),
5     booster_z = z_score(booster_5plus_pct),
6     dem_pct_2000_z = z_score(dem_pct_2000),
7     dem_pct_2020_z = z_score(dem_pct_2020)
8   )
```

across() Function

- If we want to replace our variables with z-scores, we can use the `across()` function to perform many mutations at once:

```
1 > covid_data |>
2   mutate(across(one_dose_5plus_pct:dem_pct_2020, z_score))
3
4 # A tibble: 3,114 × 8
5   fips county state one_dose_5plus_pct one_dose_65plus_pct booster_5plus_pct
6   <chr> <chr>   <chr>         <dbl>         <dbl>         <dbl>
7 1 26039 Crawford County MI          -0.508         -0.829          0.531
8 2 40015 Caddo County OK             1.40          0.843          0.439
9 3 17007 Boone County IL             0.556         0.795          0.927
10 4 12055 Highlands County FL            0.404         0.720         -0.135
11 5 34029 Ocean County NJ             0.549         0.843          0.623
12 6 01067 Henry County AL            -0.314        -0.0545        -0.799
13 7 27037 Dakota County MN             1.24          0.843          2.40
14 8 27115 Pine County MN            -0.452        -0.102          0.577
15 9 51750 Radford city VA             -1.49         -1.16         -2.47
16 10 22009 Avoyelles Parish LA            -0.231        -0.564         -0.424
17 # 3,104 more rows
18 # 2 more variables: dem_pct_2000 <dbl>, dem_pct_2020 <dbl>
19 # Use `print(n = ...)` to see more rows
```

Alternative Approach #1

- Recall, we could also target all the numeric variables:

```
1 > covid_data |>
2   mutate(across(where(is.numeric), z_score))
3
4 # A tibble: 3,114 × 8
5   fips county state one_dose_5plus_pct one_dose_65plus_pct booster_5plus_pct
6   <chr> <chr> <chr> <dbl> <dbl> <dbl>
7 1 26039 Crawford County MI -0.508 -0.829 0.531
8 2 40015 Caddo County OK 1.40 0.843 0.439
9 3 17007 Boone County IL 0.556 0.795 0.927
10 4 12055 Highlands County FL 0.404 0.720 -0.135
11 5 34029 Ocean County NJ 0.549 0.843 0.623
12 6 01067 Henry County AL -0.314 -0.0545 -0.799
13 7 27037 Dakota County MN 1.24 0.843 2.40
14 8 27115 Pine County MN -0.452 -0.102 0.577
15 9 51750 Radford city VA -1.49 -1.16 -2.47
16 10 22009 Avoyelles Parish LA -0.231 -0.564 -0.424
17 # 3,104 more rows
18 # 2 more variables: dem_pct_2000 <dbl>, dem_pct_2020 <dbl>
19 # Use `print(n = ...)` to see more rows
```


Alternative Approach #2

- We could also target only the first dose variables by pointing to variables names w/ recurring patters:

```
1 > covid_data |>
2   mutate(across(starts_with("one_dose"), z_score))
3
4 # A tibble: 3,114 × 8
5   fips county      state one_dose_5plus_pct one_dose_65plus_pct booster_5plus_pct
6   <int> <chr>      <chr>          <dbl>          <dbl>          <dbl>
7 1 26039 Crawford County MI             -0.508          -0.829          31.2
8 2 40015 Caddo County OK               1.40           0.843          30.3
9 3 17007 Boone County IL              0.556           0.795          35.1
10 4 12055 Highlands County FL             0.404           0.720          24.7
11 5 34029 Ocean County NJ              0.549           0.843          32.1
12 6 1067 Henry County AL             -0.314          -0.0545         18.2
13 7 27037 Dakota County MN              1.24           0.843          49.5
14 8 27115 Pine County MN             -0.452          -0.102          31.7
15 9 51750 Radford city VA              -1.49           -1.16           1.79
16 10 22009 Avoyelles Parish LA             -0.231          -0.564          21.9
17 # 3,104 more rows
18 # 2 more variables: dem_pct_2000 <dbl>, dem_pct_2020 <dbl>
19 # Use `print(n = ...)` to see more rows
```

Adding Arguments to Our Functions

- What if we want to be able to control `na.rm` in the calls to `mean()` and `sd()` in our `z_score()` function?
 - $\rightsquigarrow$ Add an argument!

```
1 > z_score2 <- function(x, na.rm = FALSE) {  
2   (x - mean(x, na.rm = na.rm)) / sd(x, na.rm = na.rm)  
3 }
```

- If we call the function without specifying the `na.rm`?

```
1 > head(z_score2(covid_data$one_dose_5plus_pct))  
2 [1] NA NA NA NA NA NA
```

- What about now?

```
1 > head(z_score2(covid_data$one_dose_5plus_pct, na.rm = T))  
2 [1] -0.5076545  1.3982427  0.5557809  0.4038615  0.5488754 -0.3143026
```

4/ Pivoting data longer

Mortality Data

- Data from World Bank on child mortality (under 5 yo) per 1,000 live births in different countries:

```
1 # Make sure to use read_csv() from tidyverse & readr
2 > mortality <- as_tibble(read_csv(url("https://bit.ly/3QCCPvc"))); mortality
3
4 # A tibble: 217 × 52
5   country      country_code indicator `1972` `1973` `1974` `1975` `1976` `1977` `1978`
6   <chr>         <chr>      <chr>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
7 1 Aruba        ABW        Mortalit.  NA    NA    NA    NA    NA    NA    NA
8 2 Afghanistan AFG        Mortalit.  291   285.  280.  274.  268  262.  256
9 3 Angola       AGO        Mortalit.  NA    NA    NA    NA    NA    NA    NA
10 4 Albania      ALB        Mortalit.  NA    NA    NA    NA    NA    NA    97.2
11 5 Andorra      AND        Mortalit.  NA    NA    NA    NA    NA    NA    NA
12 6 United Arab . ARE        Mortalit.  80.1  72.6  65.7  59.4  53.6  48.3  43.5
13 7 Argentina    ARG        Mortalit.  69.7  68.2  66.1  63.3  59.8  55.7  51.2
14 8 Armenia      ARM        Mortalit.  NA    NA    NA    NA    87.1  83.6  80.2
15 9 American Sam. ASM        Mortalit.  NA    NA    NA    NA    NA    NA    NA
16 10 Antigua and . ATG        Mortalit.  26.9  25.1  23.5  22.1  20.8  19.5  18.3
17 # 207 more rows
18 # 42 more variables: `1979` <dbl>, `1980` <dbl>, `1981` <dbl>, `1982` <dbl>,
19 # `1983` <dbl>, `1984` <dbl>, `1985` <dbl>, `1986` <dbl>, `1987` <dbl>, `1988` <dbl>,
20 # `1989` <dbl>, `1990` <dbl>, `1991` <dbl>, `1992` <dbl>, `1993` <dbl>, `1994` <dbl>,
21 # `1995` <dbl>, `1996` <dbl>, `1997` <dbl>, `1998` <dbl>, `1999` <dbl>, `2000` <dbl>,
22 # `2001` <dbl>, `2002` <dbl>, `2003` <dbl>, `2004` <dbl>, `2005` <dbl>, `2006` <dbl>,
23 # `2007` <dbl>, `2008` <dbl>, `2009` <dbl>, `2010` <dbl>, `2011` <dbl>, ...
24 # Use `print(n = ...)` to see more rows
```

Pivoting Longer

- Mortality data is in a “wide” format (years in columns).
 - We can convert this to country-year rows with `pivot_longer()`.
- Syntax:

```
1 mydata |>
2   pivot_longer(
3     cols = <<variables to pivot>>,
4     names_to = <<new variable to put column names>>,
5     values_to = <<new variable to put column values>>
6   )
```

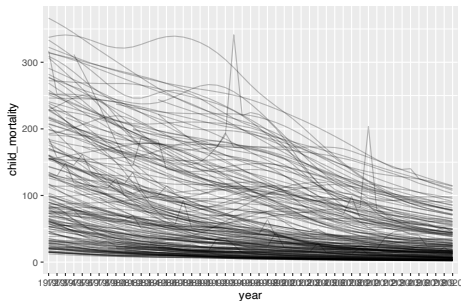
Pivoting the Mortality Data

- The 'long' data contains the years in one column `year` with corresponding mortality rates in `child_mortality`:

```
1 > mortality |>
2   select(-indicator) |>
3   pivot_longer(
4     cols = `1972`:`2020`,
5     names_to = "year",
6     values_to = "child_mortality"
7   )
8
9 # A tibble: 10,633 × 4
10   country country_code year  child_mortality
11   <chr>    <chr>      <chr>      <dbl>
12 1 Aruba   ABW         1972         NA
13 2 Aruba   ABW         1973         NA
14 3 Aruba   ABW         1974         NA
15 4 Aruba   ABW         1975         NA
16 5 Aruba   ABW         1976         NA
17 6 Aruba   ABW         1977         NA
18 7 Aruba   ABW         1978         NA
19 8 Aruba   ABW         1979         NA
20 9 Aruba   ABW         1980         NA
21 10 Aruba  ABW         1981         NA
22 # 10,623 more rows
23 # Use `print(n = ...)` to see more rows
```

Let's Do Line Plots!

```
1 > mortality |>
2   select(-indicator) |>
3   pivot_longer(
4     cols = `1972`:`2020`,
5     names_to = "year",
6     values_to = "child_mortality"
7   ) |>
8   ggplot(mapping = aes(x = year, y = child_mortality, group = country)) +
9     geom_line(alpha = 0.25)
```



- What's going on with the x-axis label?

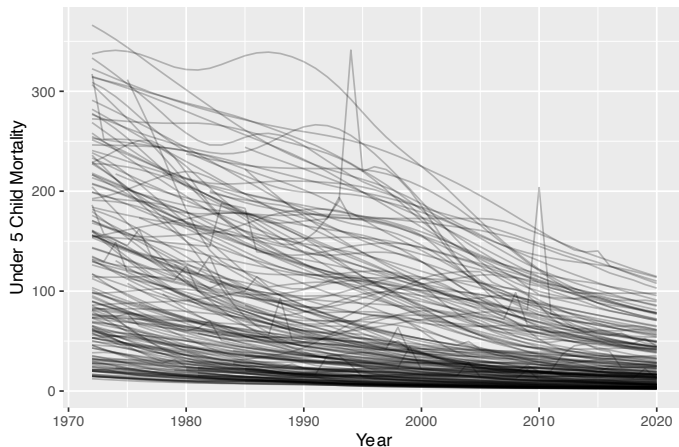
Making Sure Year is Numeric

- By default, pivoted column names are characters, but we can transform them:

```
1 > mortality_long <- mortality |>
2   select(-indicator) |>
3   pivot_longer(
4     cols = `1972`:`2020`,
5     names_to = "year",
6     values_to = "child_mortality"
7   ) |>
8   mutate(year = as.integer(year)); mortality_long
9
10 # A tibble: 10,633 × 4
11   country country_code year child_mortality
12   <chr>    <chr>      <int>      <dbl>
13 1 Aruba    ABW          1972         NA
14 2 Aruba    ABW          1973         NA
15 3 Aruba    ABW          1974         NA
16 4 Aruba    ABW          1975         NA
17 5 Aruba    ABW          1976         NA
18 6 Aruba    ABW          1977         NA
19 7 Aruba    ABW          1978         NA
20 8 Aruba    ABW          1979         NA
21 9 Aruba    ABW          1980         NA
22 10 Aruba   ABW          1981         NA
23 # 10,623 more rows
24 # Use `print(n = ...)` to see more rows
```


Let's (Re)Do Line Plots!

```
1 > mortality_long |>  
2   ggplot(mapping = aes(x = year, y = child_mortality, group = country)) +  
3   geom_line(alpha = 0.25) +  
4   labs(x = "Year", y = "Under 5 Child Mortality")
```



Spotify Data

- Data on ranks of 50 most popular songs streamed by Spotify users in the US, during 2020:

```
1 # Make sure to use read_csv() from tidyverse & readr
2 > spotify <- as_tibble(read_csv(url("https://bit.ly/49tn2qR"))); spotify
3
4 # A tibble: 490 × 54
5   `Track Name` Artist week1 week2 week3 week4 week5 week6 week7 week8 week9 week10
6   <chr>         <chr> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
7 1 The Box      Roddy. 1 1 1 1 1 1 1 1 1 1
8 2 ROXANNE      Arizo. 2 4 5 4 4 4 6 7 9 21
9 3 Yummy        Justi. 3 6 17 17 17 24 15 32 NA NA
10 4 Circles      Post . 4 7 9 10 7 10 11 10 17 26
11 5 BOP          DaBaby 5 5 7 5 11 12 18 18 32 47
12 6 Falling      Trevo. 6 8 10 7 6 8 10 11 18 28
13 7 Dance Monkey Tones. 7 13 13 12 12 13 17 13 21 33
14 8 Bandit (with. Juice. 8 11 14 14 15 20 27 26 42 NA
15 9 Futsal Shuff. Lil U. 9 9 19 21 24 32 40 49 NA NA
16 10 everything i. Billi. 10 17 28 9 8 11 14 17 29 NA
17
18 # 480 more rows
19 # 42 more variables: week11 <dbl>, week12 <dbl>, week13 <dbl>, week14 <dbl>,
20 # week15 <dbl>, week16 <dbl>, week17 <dbl>, week18 <dbl>, week19 <dbl>,
21 # week20 <dbl>, week21 <dbl>, week22 <dbl>, week23 <dbl>, week24 <dbl>,
22 # week25 <dbl>, week26 <dbl>, week27 <dbl>, week28 <dbl>, week29 <dbl>,
23 # week30 <dbl>, week31 <dbl>, week32 <dbl>, week33 <dbl>, week34 <dbl>,
24 # week35 <dbl>, week36 <dbl>, week37 <dbl>, week38 <dbl>, week39 <dbl>,
# Use `print(n = ...)` to see more rows
```

Pivoting Not Ideal

- Our previous approach isn't ideal because of the week prefix:
 - week_of_year will be a character vector which won't be ideal when we visualize/analyze the data & require additional steps!

```
1 > spotify |>
2   pivot_longer(
3     cols = c(~`Track Name`, ~Artist),
4     names_to = "week_of_year",
5     values_to = "rank"
6   )
7
8 # A tibble: 25,480 × 4
9   `Track Name` Artist      week_of_year rank
10  <chr>         <chr>      <chr>      <dbl>
11 1 The Box      Roddy Ricch week1         1
12 2 The Box      Roddy Ricch week2         1
13 3 The Box      Roddy Ricch week3         1
14 4 The Box      Roddy Ricch week4         1
15 5 The Box      Roddy Ricch week5         1
16 6 The Box      Roddy Ricch week6         1
17 7 The Box      Roddy Ricch week7         1
18 8 The Box      Roddy Ricch week8         1
19 9 The Box      Roddy Ricch week9         1
20 10 The Box     Roddy Ricch week10        1
21 # 25,470 more rows
22 # Use `print(n = ...)` to see more rows
```

- Q: What to do?

Removing a Column Name Prefix

- When the data in the column name has a fixed prefix, we can use the `names_prefix` to remove it when moving the data to rows

```
1 > spotify |>
2   pivot_longer(
3     cols = c(`Track Name`, -Artist),
4     names_to = "week_of_year",
5     values_to = "rank",
6     names_prefix = "week"
7   ) |> mutate(
8     week_of_year = as.integer(week_of_year)
9   )
10
11 # A tibble: 25,480 × 4
12   `Track Name` Artist      week_of_year rank
13   <chr>         <chr>         <int> <dbl>
14 1 The Box      Roddy Ricch          1     1
15 2 The Box      Roddy Ricch          2     1
16 3 The Box      Roddy Ricch          3     1
17 4 The Box      Roddy Ricch          4     1
18 5 The Box      Roddy Ricch          5     1
19 6 The Box      Roddy Ricch          6     1
20 7 The Box      Roddy Ricch          7     1
21 8 The Box      Roddy Ricch          8     1
22 9 The Box      Roddy Ricch          9     1
23 10 The Box     Roddy Ricch         10     1
24 # 25,470 more rows
25 # Use `print(n = ...)` to see more rows
```

5/ Joining data sets

Revisiting Mortality Data

- Data from World Bank on child mortality (under 5 yo) per 1,000 live births in different countries:

```
1 # Make sure to use read_csv() from tidyverse & readr
2 > mortality <- as_tibble(read_csv(url("https://bit.ly/3QCCPvc"))); mortality
3
4 # A tibble: 217 × 52
5   country      country_code indicator `1972` `1973` `1974` `1975` `1976` `1977` `1978`
6   <chr>         <chr>         <chr>   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
7 1 Aruba        ABW          Mortalit.  NA    NA    NA    NA    NA    NA    NA
8 2 Afghanistan AFG          Mortalit.  291   285.  280.  274.  268   262.  256
9 3 Angola       AGO          Mortalit.  NA    NA    NA    NA    NA    NA    NA
10 4 Albania      ALB          Mortalit.  NA    NA    NA    NA    NA    NA    97.2
11 5 Andorra      AND          Mortalit.  NA    NA    NA    NA    NA    NA    NA
12 6 United Arab . ARE          Mortalit.  80.1  72.6  65.7  59.4  53.6  48.3  43.5
13 7 Argentina    ARG          Mortalit.  69.7  68.2  66.1  63.3  59.8  55.7  51.2
14 8 Armenia      ARM          Mortalit.  NA    NA    NA    NA    87.1  83.6  80.2
15 9 American Sam. ASM          Mortalit.  NA    NA    NA    NA    NA    NA    NA
16 10 Antigua and . ATG          Mortalit.  26.9  25.1  23.5  22.1  20.8  19.5  18.3
17 # 207 more rows
18 # 42 more variables: `1979` <dbl>, `1980` <dbl>, `1981` <dbl>, `1982` <dbl>,
19 #   `1983` <dbl>, `1984` <dbl>, `1985` <dbl>, `1986` <dbl>, `1987` <dbl>, `1988` <dbl>,
20 #   `1989` <dbl>, `1990` <dbl>, `1991` <dbl>, `1992` <dbl>, `1993` <dbl>, `1994` <dbl>,
21 #   `1995` <dbl>, `1996` <dbl>, `1997` <dbl>, `1998` <dbl>, `1999` <dbl>, `2000` <dbl>,
22 #   `2001` <dbl>, `2002` <dbl>, `2003` <dbl>, `2004` <dbl>, `2005` <dbl>, `2006` <dbl>,
23 #   `2007` <dbl>, `2008` <dbl>, `2009` <dbl>, `2010` <dbl>, `2011` <dbl>, ...
24 # Use `print(n = ...)` to see more rows
```

Gapminder Data

- Recall we have access to life expectancy, population size, and income level (i.e., GDP per Cap) at the country level!
 - We may want to enrich our analysis with these additional information.

```
1 > require(gapminder)
2 > gapminder
3
4 # A tibble: 1,704 × 6
5   country    continent  year lifeExp      pop gdpPercap
6   <fct>      <fct>      <int> <dbl>    <int>    <dbl>
7 1 Afghanistan Asia      1952  28.8  8425333  779.
8 2 Afghanistan Asia      1957  30.3  9240934  821.
9 3 Afghanistan Asia      1962  32.0 10267083  853.
10 4 Afghanistan Asia      1967  34.0 11537966  836.
11 5 Afghanistan Asia      1972  36.1 13079460  740.
12 6 Afghanistan Asia      1977  38.4 14880372  786.
13 7 Afghanistan Asia      1982  39.9 12881816  978.
14 8 Afghanistan Asia      1987  40.8 13867957  852.
15 9 Afghanistan Asia      1992  41.7 16317921  649.
16 10 Afghanistan Asia      1997  41.8 22227415  635.
17 # 1,694 more rows
18 # Use `print(n = ...)` to see more rows
```

Joining Data Sets

- What if we want to add the `child_mortality` variable to the `gapminder` data?
- Just add the columns (e.g. `cbind()`)?
 - Issue: rows are not aligned properly!

```
1 > gapminder |>
2   select(country, year) |>
3   head(10)
4
5 # A tibble: 10 × 2
6   country      year
7   <fct>      <int>
8 1 Afghanistan 1952
9 2 Afghanistan 1957
10 3 Afghanistan 1962
11 4 Afghanistan 1967
12 5 Afghanistan 1972
13 6 Afghanistan 1977
14 7 Afghanistan 1982
15 8 Afghanistan 1987
16 9 Afghanistan 1992
17 10 Afghanistan 1997
```

```
1 > mortality_long |>
2   select(country, year) |>
3   head(10)
4
5 # A tibble: 10 × 2
6   country      year
7   <chr>      <int>
8 1 Aruba      1972
9 2 Aruba      1973
10 3 Aruba      1974
11 4 Aruba      1975
12 5 Aruba      1976
13 6 Aruba      1977
14 7 Aruba      1978
15 8 Aruba      1979
16 9 Aruba      1980
17 10 Aruba      1981
```


Key Variables

- A **primary key** is a variable or set of variables that uniquely identifies rows in the data.
 - {country, year} in the gapminder data
- A **foreign key** is the corresponding variable(s) in another table.
 - {country, year} in the mortality_long data
- If we 'align' the two tables based on these variables, we can add variables from one to the other (i.e., join them).

Checking if the Keys are Unique

- Joining won't work properly if these keys are not unique. Let's check.

Checking primary key is unique:

```
1 > gapminder |>
2   count(country, year) |>
3   filter(n > 1)
4
5 # A tibble: 0 × 3
6 #   3 variables: country <fct>,
7 #   year <int>, n <int>
```

Checking foreign key:

```
1 > mortality_long |>
2   count(country, year) |>
3   filter(n > 1)
4
5 # A tibble: 0 × 3
6 #   3 variables: country <chr>,
7 #   year <int>, n <int>
```

- No duplicate rows based on the combination of 'country' and 'year' in the datasets.
 - Each pair of 'country' and 'year' appears only once, serving as a unique identifier for each observation.

left_join(): Add Variables to Primary Data

- `left_join()` keeps all rows from the first primary/piped data:

```
1 > gapminder |>
2   left_join(mortality_long) |>
3   select(country, year, lifeExp, pop, gdpPercap, child_mortality) |>
4   head(n = 6)
5
6 Joining with `by = join_by(country, year)`
7 # A tibble: 6 × 6
8   country      year lifeExp      pop gdpPercap child_mortality
9   <chr>      <int>   <dbl>   <int>   <dbl>         <dbl>
10 1 Afghanistan 1952    28.8  8425333    779.          NA
11 2 Afghanistan 1957    30.3  9240934    821.          NA
12 3 Afghanistan 1962    32.0 10267083    853.          NA
13 4 Afghanistan 1967    34.0 11537966    836.          NA
14 5 Afghanistan 1972    36.1 13079460    740.         291
15 6 Afghanistan 1977    38.4 14880372    786.         262.
```

- Rows in primary table that are not in foreign table?
 - Stored as **NA**!

inner_join(): add and filter

- `inner_join()` adds the variables from the foreign data and filters to rows in both datasets:

```
1 > gapminder |>
2   inner_join(mortality_long) |>
3   select(country, year, lifeExp, pop, gdpPercap, child_mortality) |>
4   head(n = 6)
5
6 Joining with `by = join_by(country, year)`
7 # A tibble: 6 × 6
8   country      year lifeExp      pop gdpPercap child_mortality
9   <chr>      <int> <dbl>    <int>    <dbl>      <dbl>
10 1 Afghanistan  1972   36.1 13079460    740.        291
11 2 Afghanistan  1977   38.4 14880372    786.        262.
12 3 Afghanistan  1982   39.9 12881816    978.        231.
13 4 Afghanistan  1987   40.8 13867957    852.        198.
14 5 Afghanistan  1992   41.7 16317921    649.        166.
15 6 Afghanistan  1997   41.8 22227415    635.        142.
```

- R automatically determined the key variables.
 - Ideally, we should specify the joining/key variables by setting `by = c("country", "year")` in the join fns.

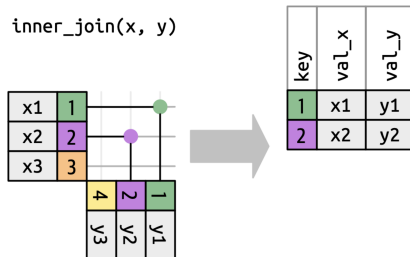
How Inner Joins Work

- Two data sets¹:

x		y	
1	x1	1	y1
2	x2	2	y2
3	x3	4	y3

- Find matching keys:

`inner_join(x, y)`



¹Credit: R for Data Science

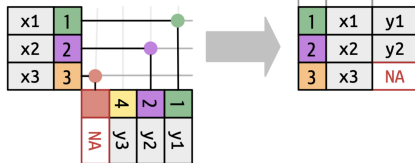
How Left Joins Work

- Two data sets²:

x		y	
1	x1	1	y1
2	x2	2	y2
3	x3	4	y3

- Keep all x keys (obs. in primary data):

`left_join(x, y)`



²Credit: R for Data Science

More Complicated Example: Flight Data

```
1 > library(nycflights13)
2 > flights2 <- flights |>
3   select(year, time_hour, origin, dest, tailnum, carrier); flights2
4
5 # A tibble: 336,776 × 6
6   year time_hour      origin dest tailnum carrier
7   <int> <dtm>      <chr>  <chr> <chr>  <chr>
8 1 2013 2013-01-01 05:00:00 EWR   IAH   N14228  UA
9 2 2013 2013-01-01 05:00:00 LGA   IAH   N24211  UA
10 3 2013 2013-01-01 05:00:00 JFK   MIA   N619AA  AA
11 4 2013 2013-01-01 05:00:00 JFK   BQN   N804JB  B6
12 5 2013 2013-01-01 06:00:00 LGA   ATL   N668DN  DL
13 6 2013 2013-01-01 05:00:00 EWR   ORD   N39463  UA
14 7 2013 2013-01-01 06:00:00 EWR   FLL   N516JB  B6
15 8 2013 2013-01-01 06:00:00 LGA   IAD   N829AS  EV
16 9 2013 2013-01-01 06:00:00 JFK   MCO   N593JB  B6
17 10 2013 2013-01-01 06:00:00 LGA   ORD   N3ALAA  AA
18 # 336,766 more rows
19 # Use `print(n = ...)` to see more rows
```

Planes Data

```
1 > planes2 <- planes |>
2   select(tailnum, year, type, engine, seats); planes2
3
4 # A tibble: 3,322 × 5
5   tailnum year type engine seats
6   <chr>   <int> <chr>   <chr>   <int>
7 1 N10156  2004 Fixed wing multi engine Turbo-fan 55
8 2 N102UW  1998 Fixed wing multi engine Turbo-fan 182
9 3 N103US  1999 Fixed wing multi engine Turbo-fan 182
10 4 N104UW  1999 Fixed wing multi engine Turbo-fan 182
11 5 N10575  2002 Fixed wing multi engine Turbo-fan 55
12 6 N105UW  1999 Fixed wing multi engine Turbo-fan 182
13 7 N107US  1999 Fixed wing multi engine Turbo-fan 182
14 8 N108UW  1999 Fixed wing multi engine Turbo-fan 182
15 9 N109UW  1999 Fixed wing multi engine Turbo-fan 182
16 10 N110UW 1999 Fixed wing multi engine Turbo-fan 182
17 # 3,312 more rows
18 # Use `print(n = ...)` to see more rows
```


What Happens with 'Naive' Join?

```
1 > flights2 |>
2   left_join(planes2)
3
4 Joining with `by = join_by(year, tailnum)`
5 # A tibble: 336,776 × 9
6   year time_hour      origin dest  tailnum carrier type  engine seats
7   <int> <dtm>      <chr>  <chr> <chr>  <chr>  <chr> <chr>  <int>
8 1 2013 2013-01-01 05:00:00 EWR    IAH  N14228  UA    NA    NA    NA
9 2 2013 2013-01-01 05:00:00 LGA    IAH  N24211  UA    NA    NA    NA
10 3 2013 2013-01-01 05:00:00 JFK    MIA  N619AA  AA    NA    NA    NA
11 4 2013 2013-01-01 05:00:00 JFK    BQN  N804JB  B6    NA    NA    NA
12 5 2013 2013-01-01 06:00:00 LGA    ATL  N668DN  DL    NA    NA    NA
13 6 2013 2013-01-01 05:00:00 EWR    ORD  N39463  UA    NA    NA    NA
14 7 2013 2013-01-01 06:00:00 EWR    FLL  N516JB  B6    NA    NA    NA
15 8 2013 2013-01-01 06:00:00 LGA    IAD  N829AS  EV    NA    NA    NA
16 9 2013 2013-01-01 06:00:00 JFK    MCO  N593JB  B6    NA    NA    NA
17 10 2013 2013-01-01 06:00:00 LGA    ORD  N3ALAA  AA    NA    NA    NA
18 # 336,766 more rows
19 # Use `print(n = ...)` to see more rows
```

- As usual, R automatically detects which columns should be used as key variables $\rightsquigarrow$ `c(year, tailnum)` – Wait..
 - year in flights is flight year vs. the manufacturing year for the airplane in planes!

Specify the Joining Variables

```
1 > flights2 |>
2   left_join(planes2, by = "tailnum")
3
4 # A tibble: 336,776 × 10
5   year.x time_hour      origin dest tailnum carrier year.y type engine seats
6   <int> <dtm>         <chr> <chr> <chr> <chr> <int> <chr> <chr> <int>
7 1 2013 2013-01-01 05:00:00 EWR   IAH   N14228 UA      1999 Fixe. Turbo. 149
8 2 2013 2013-01-01 05:00:00 LGA   IAH   N24211 UA      1998 Fixe. Turbo. 149
9 3 2013 2013-01-01 05:00:00 JFK   MIA   N619AA AA      1990 Fixe. Turbo. 178
10 4 2013 2013-01-01 05:00:00 JFK   BQN   N804JB B6      2012 Fixe. Turbo. 200
11 5 2013 2013-01-01 06:00:00 LGA   ATL   N668DN DL      1991 Fixe. Turbo. 178
12 6 2013 2013-01-01 05:00:00 EWR   ORD   N39463 UA      2012 Fixe. Turbo. 191
13 7 2013 2013-01-01 06:00:00 EWR   FLL   N516JB B6      2000 Fixe. Turbo. 200
14 8 2013 2013-01-01 06:00:00 LGA   IAD   N829AS EV      1998 Fixe. Turbo. 55
15 9 2013 2013-01-01 06:00:00 JFK   MCO   N593JB B6      2004 Fixe. Turbo. 200
16 10 2013 2013-01-01 06:00:00 LGA   ORD   N3ALAA AA      NA NA    NA    NA
17 # 336,766 more rows
18 # Use `print(n = ...)` to see more rows
```

Or Change Variable Names

```
1 > flights2 |>
2   left_join(planes2 |>
3     rename(manufacture_year = year))
4
5 Joining with `by = join_by(tailnum)`
6 # A tibble: 336,776 × 10
7   year time_hour      origin dest tailnum carrier manufacture_year type
8   <int> <dtm>      <chr>  <chr> <chr>  <chr>      <int> <chr>
9 1 2013 2013-01-01 05:00:00 EWR   IAH   N14228  UA      1999 Fixed wi.
10 2 2013 2013-01-01 05:00:00 LGA   IAH   N24211  UA      1998 Fixed wi.
11 3 2013 2013-01-01 05:00:00 JFK   MIA   N619AA  AA      1990 Fixed wi.
12 4 2013 2013-01-01 05:00:00 JFK   BQN   N804JB  B6      2012 Fixed wi.
13 5 2013 2013-01-01 06:00:00 LGA   ATL   N668DN  DL      1991 Fixed wi.
14 6 2013 2013-01-01 05:00:00 EWR   ORD   N39463  UA      2012 Fixed wi.
15 7 2013 2013-01-01 06:00:00 EWR   FLL   N516JB  B6      2000 Fixed wi.
16 8 2013 2013-01-01 06:00:00 LGA   IAD   N829AS  EV      1998 Fixed wi.
17 9 2013 2013-01-01 06:00:00 JFK   MCO   N593JB  B6      2004 Fixed wi.
18 10 2013 2013-01-01 06:00:00 LGA   ORD   N3ALAA  AA      NA NA
19 # 336,766 more rows
20 # 2 more variables: engine <chr>, seats <int>
21 # Use `print(n = ...)` to see more rows
```

Enjoy Your Weekends! :)

Contact Information:

jaewon.yoo@iss.nthu.edu.tw

<https://j1yoo.github.io/>



Code for Visualization

```
1 ##### Week 7: Bivariate Relationships and Tidying Data
2 ## Slide 4/55:
3 pacman::p_load(tidyverse); covid_data <- as_tibble(read.csv(url("https://bit.ly/3F0Lohq")))
4
5 # Pivot data longer:
6 covid_long <- covid_data |>
7   pivot_longer(
8     cols = c(one_dose_5plus_pct, one_dose_65plus_pct),
9     names_to = "population",
10    values_to = "one_dose_pct"
11  ) |>
12   mutate(Population = if_else(population == "one_dose_5plus_pct", "Ages 5+", "Ages 65+"))
13
14 ## Slide 5/55:
15 ggplot(data = covid_long, aes(x = one_dose_pct, fill = Population)) +
16   geom_histogram(alpha = .5) +
17   geom_vline(xintercept = 60) +
18   labs(x = "Percent of Population with at least One Does", y = "Count",
19        )
20
21 ## Slide 7/55:
22 ggplot(data = covid_long, aes(x = one_dose_pct, fill = Population)) +
23   geom_histogram(alpha = .5) +
24   geom_vline(aes(xintercept = 60, color = Population)) +
25   labs(x = "Percent of Population with at least One Does", y = "Count",
26        )
```

Code for Visualization Cont.

```
1 ## Slide 8/55:
2 centered_60 <- covid_long |>
3   group_by(Population) |>
4   summarize(centered_60 = 60 - mean(one_dose_pct, na.rm = T)); centered_60
5
6 covid_long |>
7   group_by(Population) |>
8   mutate(one_dose_centered = one_dose_pct - mean(one_dose_pct, na.rm = T),
9          centered_60 = 60 - mean(one_dose_pct, na.rm = T)) |>
10  ggplot(aes(x = one_dose_centered, fill = Population)) +
11  geom_histogram(alpha = .5) +
12  geom_vline(aes(xintercept = centered_60, color = Population)) +
13  labs(x = "Percent of Population with at least One Does", y = "Count",
14       )
15
16 ## Slide 9/55:
17 scaled_60 <- covid_long |>
18   group_by(Population) |>
19   summarize(scaled_60 = (60 - mean(one_dose_pct, na.rm = T))/
20            sd(one_dose_pct, na.rm = T)); scaled_60
21
22 covid_long |>
23   group_by(Population) |>
24   mutate(one_dose_z = (one_dose_pct - mean(one_dose_pct, na.rm = T))/
25          sd(one_dose_pct, na.rm = T),
26          scaled_60 = (60 - mean(one_dose_pct, na.rm = T))/
27          sd(one_dose_pct, na.rm = T)) |>
28  ggplot(aes(x = one_dose_z, fill = Population)) +
29  geom_histogram(alpha = .5) +
30  geom_vline(aes(xintercept = scaled_60, color = Population))
```